AI Tools Catalogue — a Knowledge Graph of AI Tools, Models and Providers

Semestral project, 4IZ444 Knowledge Graphs and Linked Data

Ota Vlna

Matěj Fadrhons

Prague University of Economics and Business, winter semester 2026/2027

Contents

1 Domain description
  1.1 Motivation
  1.2 Use case and users
  1.3 Competency questions
2 Ontology design
  2.1 Core concepts
  2.2 Axioms and constraints
  2.3 Conceptual model
  2.4 Reuse of existing ontologies
3 Data
4 SPARQL queries
  CQ1 — Coding tools available in Visual Studio Code
  CQ2 — Models with image input and text output
  CQ3 — Average price of paid plans per provider
  CQ4 — Best open-weights models on MMLU
  CQ5 — Number of tools per task
  CQ6 — Tools built on models of other providers
  CQ7 — Model lineage
  CQ8 — Largest context window per provider
  CQ9 — Providers offering both text and image generation
  CQ10 — Free tools running on openly licensed models
  Q11 — Providers' best scores compared with the benchmark average
  Q12 — Analysis of model names (string functions)
  Q13 — Competitor network (symmetric property, EXISTS)
  Q14 — Models used in tools but lacking benchmark results (negation)
  Q15 — CONSTRUCT: materialising tool dependencies
5 SHACL validation
  S1 AIToolShape
  S2 AIModelShape
  S3 LanguageModelShape and ImageGenerationModelShape
  S4 BenchmarkResultShape
  S5 PricingPlanShape
  S6 ModelLineageShape (SPARQL constraint)
  5.1 Validation results
6 Conclusion
Appendix A — Repository and reproducibility

1 Domain description

1.1 Motivation

Over the last three years the number of AI products has exploded. A single company typically offers several *tools* (e.g. OpenAI offers ChatGPT and Codex; Anthropic offers Claude and Claude Code), every tool can use several *models*, and the same model is often embedded in tools of completely different companies (GitHub Copilot, Cursor and Perplexity all let the user choose between models from OpenAI, Anthropic and Google). On top of that, each tool has its own pricing tiers, integrations (IDE plug-ins, Slack, mobile apps, REST API) and each model is described by licences, modalities, context-window sizes and benchmark scores that are scattered over vendor blogs, model cards and leaderboards.

This information is naturally a **graph**: tools, models, providers, licences and benchmarks are interlinked entities, and the interesting questions are about paths through them ("which free tools run on an open-weights model that scores well on MMLU?"). A spreadsheet or a relational table per entity type quickly becomes awkward for such questions, whereas a knowledge graph handles them with a single SPARQL query.

1.2 Use case and users

The knowledge graph serves as a **catalogue and decision-support tool for selecting AI tools**. Typical users are:

- **IT / procurement departments** comparing tools by price, integrations and licensing (e.g. "we need a coding assistant for VS Code and JetBrains under 25 USD per user"),
- **software developers** choosing a coding assistant or a model to run locally,
- **analysts and journalists** observing the market — which providers depend on whose models, how quickly model generations follow each other, how open-weights models compare to closed ones.

The graph is a snapshot of the market around autumn 2025. It was created manually from public sources (vendor pricing pages, model cards, announcements); benchmark scores are mostly vendor-reported and were measured under different settings, so they are indicative and serve to demonstrate analytical queries rather than as an authoritative leaderboard.

1.3 Competency questions

The knowledge graph must be able to answer the following competency questions (CQ). Questions CQ3, CQ5, CQ6, CQ8 and CQ11 have an analytical character (aggregation, comparison).

ID	Competency question
CQ1	Which AI tools support code generation and can be used inside Visual Studio Code, and who provides them?
CQ2	Which AI models accept images as input and produce text, who develops them and in how many tools are they used?
CQ3	What is the average, minimum and maximum monthly price of paid plans per provider?
CQ4	Which openly licensed (open-weights) models perform best on MMLU, and how large are they?
CQ5	How many AI tools support each task, and which ones?
CQ6	Which AI tools are powered by models of a *different* provider than the one offering the tool?
CQ7	What is the complete lineage (all predecessors) of a given model and how far apart were the releases?
CQ8	Which language model has the largest context window for each provider?
CQ9	Which providers offer both text generation and image generation, and with which in-house image models?
CQ10	Which AI tools can be used for free and run on at least one openly licensed model?

In addition, five further queries (Q11–Q15) exercise more advanced SPARQL features (nested aggregation, string functions, symmetric property paths with `EXISTS`, negation with `FILTER NOT EXISTS`, and `CONSTRUCT`); see Section 4.

2 Ontology design

2.1 Core concepts

From the competency questions we identified the following core concepts. The central design decision is the separation of an **AI tool** (a product a user interacts with — ChatGPT) from an **AI model** (a trained model — GPT-5). Without this distinction questions CQ2, CQ6, CQ7 or CQ10 cannot be expressed. Two further concepts were added on top of the initially proposed eight:

- **BenchmarkResult** — a score is a property of the *pair* (model, benchmark), not of the model or the benchmark alone. We therefore model it as an n-ary relation (a reified "evaluation" node), following the W3C pattern *Defining N-ary Relations on the Semantic Web* and the `mls:ModelEvaluation` class of the W3C ML Schema.
- **PricingPlan** — needed for price comparisons (CQ3, CQ10). A plan is a separate entity because one plan can cover several tools (e.g. the *Claude Pro* subscription covers both Claude and Claude Code, *ChatGPT Plus* covers ChatGPT and Codex).

The namespace of the ontology is `https://otavlna.github.io/4iz444-ai-tools-kg/ontology#` (prefix `ait:`), instance data use `https://otavlna.github.io/4iz444-ai-tools-kg/data/` (prefix `aid:`).

Classes (13)

Class	Superclass	Description
`ait:AITool`	`schema:SoftwareApplication`	End-user AI product (ChatGPT, Claude Code, Midjourney, …).
`ait:AIModel`	`mls:Model`	A specific trained model release (GPT-5, Claude Sonnet 4.5, …).
`ait:LanguageModel`	`ait:AIModel`	Model whose primary output is text/code.
`ait:ImageGenerationModel`	`ait:AIModel`	Model whose primary output is images.
`ait:Provider`	`schema:Organization`	Organisation developing models and/or offering tools.
`ait:Task`	— (close match `mls:Task`)	Kind of work: code generation, summarisation, …
`ait:Modality`	—	Data type consumed/produced: text, image, audio, video.
`ait:License`	`dcterms:LicenseDocument`	Licence under which a model is available.
`ait:OpenLicense`	`ait:License`	Licence allowing download and use of the weights (open-weights).
`ait:Integration`	—	Platform or interface the tool is available in (VS Code, Slack, REST API, …).
`ait:Benchmark`	`schema:Dataset`	Standardised evaluation (MMLU, SWE-bench Verified, …).
`ait:BenchmarkResult`	`mls:ModelEvaluation`	Score of one model on one benchmark (n-ary relation).
`ait:PricingPlan`	`schema:Offer`	Subscription tier with a monthly price.

Object properties (15)

Property	Domain → Range	Characteristics	Description
`ait:providedBy`	AITool → Provider	functional; ⊑ `schema:provider`	Organisation that offers the tool.
`ait:poweredBy`	AITool → AIModel		Model used by the tool.
`ait:developedBy`	AIModel → Provider	inverse of `ait:develops`; ⊑ `schema:creator`	Organisation that trained the model.
`ait:develops`	Provider → AIModel	inverse of `ait:developedBy`	Models released by the provider.
`ait:supportsTask`	AITool → Task		Task the tool can be used for.
`ait:inputModality`	AIModel → Modality		Accepted input modality.
`ait:outputModality`	AIModel → Modality		Produced output modality.
`ait:hasLicense`	AIModel → License	⊑ `dcterms:license`	Licence of the model.
`ait:integratesWith`	AITool → Integration		Platform in which the tool is available.

Property	Domain → Range	Characteristics	Description
`ait:hasPricingPlan`	AITool → PricingPlan		Plan under which the tool is offered.
`ait:evaluatedModel`	BenchmarkResult → AIModel	functional	Model the result belongs to.
`ait:onBenchmark`	BenchmarkResult → Benchmark	functional	Benchmark of the result.
`ait:measuresTask`	Benchmark → Task		Task the benchmark measures.
`ait:derivedFrom`	AIModel → AIModel	**transitive**	Successor/derivative of an earlier model.
`ait:competesWith`	AITool → AITool	**symmetric, irreflexive**	Direct market competitors.

Datatype properties (10)

Property	Domain → Range	Characteristics	Description
`ait:releaseDate`	AITool ⊔ AIModel → `xsd:date`	functional; ⊑ `dcterms:issued`	First public availability.
`ait:contextWindow`	LanguageModel → `xsd:integer`	functional	Maximum tokens per request.
`ait:parameterCount`	AIModel → `xsd:decimal`	functional	Parameters in billions (if published).
`ait:score`	BenchmarkResult → `xsd:decimal`	functional	Score in percent (0–100).
`ait:metric`	Benchmark → `xsd:string`	functional	Metric of the benchmark (accuracy, pass@1, …).
`ait:monthlyPrice`	PricingPlan → `xsd:decimal`	functional	Price per user per month, 0 = free.
`ait:currency`	PricingPlan → `xsd:string`	functional	ISO 4217 currency code.
`ait:foundedYear`	Provider → `xsd:gYear`	functional	Year the provider was founded.
`ait:headquartersCountry`	Provider → `xsd:string`		Country of the headquarters.
`ait:spdxId`	License → `xsd:string`	functional	SPDX licence identifier.

Names of all entities are given by `rdfs:label`; homepages by the reused `foaf:homepage`. Every class and property of the ontology has an `rdfs:label` and an `rdfs:comment` (checked automatically by the build script).

2.2 Axioms and constraints

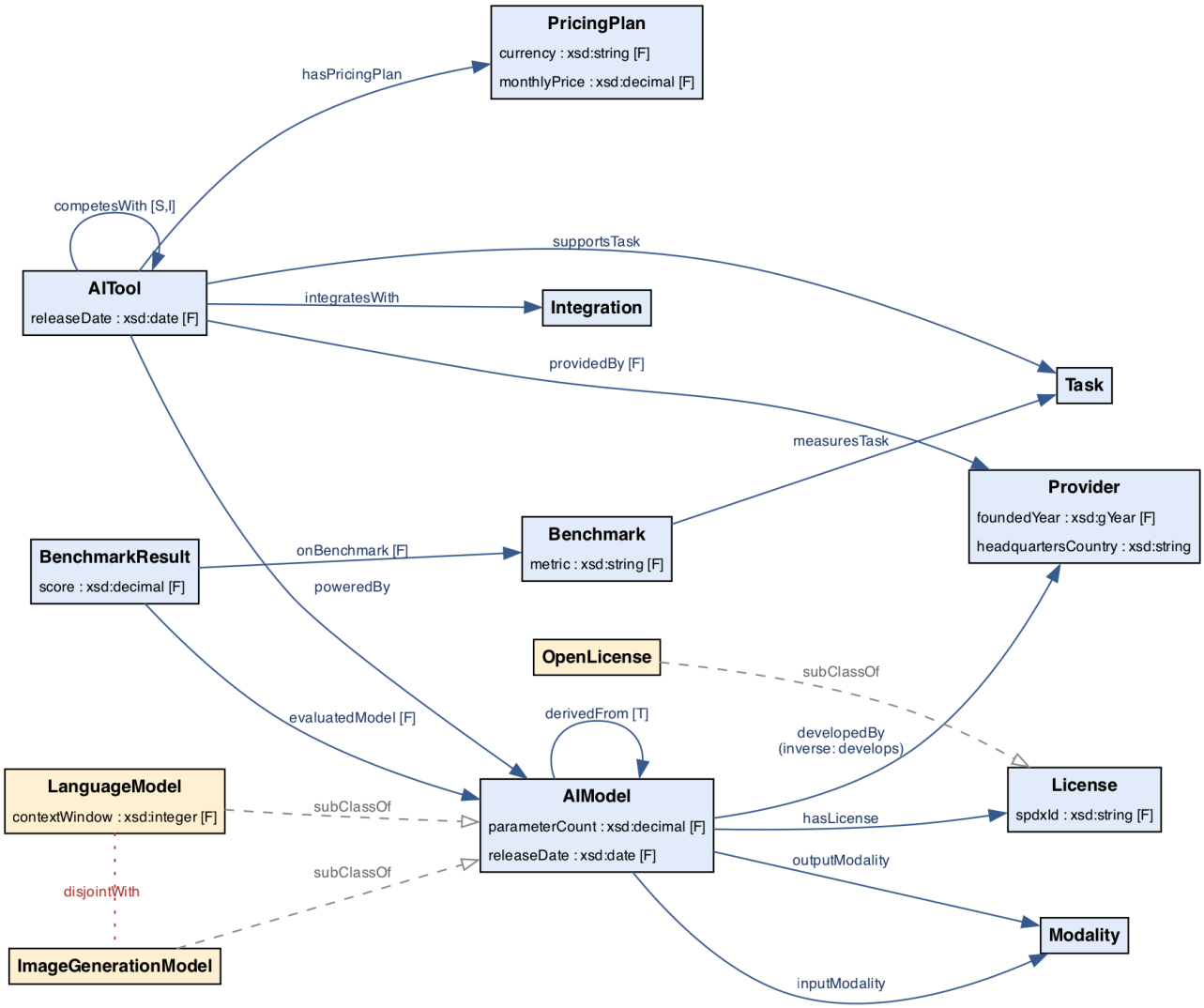
Besides domain and range restrictions for all 25 properties, the ontology contains:

- **class hierarchy** — `LanguageModel ⊑ AIModel`, `ImageGenerationModel ⊑ AIModel`, `OpenLicense ⊑ License`, plus alignment of all main classes to external superclasses (schema.org, ML Schema, Dublin Core);
- **disjointness** — `owl:AllDisjointClasses` over the ten top-level classes (a tool cannot be a model, a provider cannot be a task, …) and `LanguageModel owl:disjointWith ImageGenerationModel`;
- **property characteristics** — functional properties (`providedBy`, `evaluatedModel`, `onBenchmark`, `releaseDate`, `score`, `monthlyPrice`, …), an inverse pair (`developedBy`/`develops`), a transitive property (`derivedFrom` — the lineage of a model), a symmetric and irreflexive property (`competesWith`);
- **union domain** — `releaseDate` is applicable to tools and models (`rdfs:domain [ owl:unionOf (ait:AITool ait:AIModel) ]`);
- **sub-properties** of reused vocabularies (`hasLicense ⊑ dcterms:license`, `releaseDate ⊑ dcterms:issued`, `providedBy ⊑ schema:provider`, `developedBy ⊑ schema:creator`).

`competesWith` is deliberately only symmetric+irreflexive and `derivedFrom` only transitive — OWL 2 DL forbids irreflexivity on transitive (non-simple) properties, so the "a model cannot be derived from itself / from a newer model" rule is enforced by SHACL instead (shape S6).

The ontology is valid Turtle (all reused external terms are explicitly declared so that it can be loaded into Protégé as OWL 2) and is free of logical inconsistencies: OWL 2 RL closure (owlrl 7.6.2) over ontology + data inferred 1657 additional triples; **0 inconsistencies** found (disjoint-class membership, `owl:Nothing`, irreflexive and functional property violations).

2.3 Conceptual model



Conceptual model of the AI Tools Catalogue ontology. Blue boxes are top-level classes, yellow boxes are subclasses (dashed arrows = `rdfs:subClassOf`). Solid arrows are object properties from domain to range; datatype properties are listed inside the classes. Characteristics: F = functional, T = transitive, S = symmetric, I = irreflexive.

The diagram is generated automatically from the Turtle file (`scripts/build.py`), so it always corresponds to the actual ontology.

2.4 Reuse of existing ontologies

We searched Linked Open Vocabularies (LOV), schema.org and W3C specifications for terms covering our concepts. The most relevant candidates were:

Vocabulary	Relevant terms	Fit
schema.org	`SoftwareApplication`, `Organization`, `Offer`, `price`, `priceCurrency`, `provider`, `creator`, `Dataset`, `isBasedOn`, `sameAs`	Very good for generic products, organisations and offers; no notion of AI models, modalities or benchmarks.
W3C ML Schema (`mls:`)	`Model`, `ModelEvaluation`, `EvaluationMeasure`, `Task`, `Implementation`, `Software`	Designed for ML experiments; models and evaluations map well. Its `Task` means an ML task (classification, regression), which is only close to our user-facing task.
Dublin Core Terms	`license`, `LicenseDocument`, `issued`, `created`, `creator`, `title`	Standard metadata; ideal for licences and dates.
FOAF	`Organization`, `homepage`	Widely used, `homepage` directly reusable.

Vocabulary	Relevant terms	Fit
SKOS	`Concept`, `ConceptScheme`, `broader`, `exactMatch`, `closeMatch`	Suitable for controlled lists (tasks, modalities) and for alignments.
SPDX	licence list and identifiers	Standard identifiers of licences (Apache-2.0, MIT).
Wikidata	entities for providers (OpenAI Q21708200, Google Q95, Meta Q380, GitHub Q364)	Useful for linking instances to the LOD cloud.
MEX vocabulary, ML-Onto, DCAT-AP for AI	ML experiment metadata	Too detailed (hyper-parameters, runs) or not maintained.

Decision. We define our **own core classes and properties** in the `ait:` namespace and **align** them to the external vocabularies instead of using the external terms directly. Reasons:

1. *Precise semantics and constraints.* We need domain/range restrictions, disjointness and functional properties (e.g. “a tool has exactly one provider”). Asserting such axioms on `schema:provider` or `mls:Model` would change the meaning of terms we do not own and would conflict with their intentionally loose definitions (schema.org uses `domainIncludes` / `rangeIncludes` instead of strict domains).
2. *Missing concepts.* No vocabulary covers the distinction tool vs. model, modalities, context windows or pricing plans shared by several products.
3. *Interoperability is kept* via `rdfs:subClassOf` (e.g. `ait:AITool ⊑ schema:SoftwareApplication`, `ait:BenchmarkResult ⊑ mls:ModelEvaluation`), `rdfs:subPropertyOf` (`ait:hasLicense ⊑ dcterms:license`, `ait:releaseDate ⊑ dcterms:issued`) and `skos:closeMatch` (`ait:Task ≈ mls:Task`). A consumer that understands schema.org or Dublin Core can therefore still interpret our data after RDFS reasoning.

Some terms are **reused directly** because their semantics fits exactly and no extra constraints are needed: `rdfs:label` (names), `foaf:homepage` (homepages), `schema:sameAs` (links of providers to Wikidata), `schema:isBasedOn` (output of the CONSTRUCT query Q15), Dublin Core and VANN for the ontology metadata, and SPDX identifiers as values of `ait:spdxId`.

3 Data

The knowledge graph (`data/ai-tools-data.ttl`) was created manually with the help of an LLM, based on public information (vendor pricing pages, model cards and announcements) and then reviewed. It covers 15 tools from 12 providers and 27 models, including the tools named in the original topic (ChatGPT, Claude, Gemini, Midjourney, GitHub Copilot) and representative open-weights models (Llama, DeepSeek, Qwen, Mistral, Gemma, gpt-oss). All classes and all properties (except `ait:develops`, which is the inverse of `ait:developedBy` and is inferred by the reasoner) are used in the data.

Metric	Value
Triples in the ontology	254
Triples in the knowledge graph (data)	987
Classes defined (`ait:`)	13
Object properties defined	15
Datatype properties defined	10
`ait:` properties used in data	24 of 25
Distinct subjects in data	170

Class (incl. subclasses)	Instances
`ait:BenchmarkResult`	44
`ait:PricingPlan`	34
`ait:AIModel`	27
`ait:LanguageModel`	22
`ait:AITool`	15
`ait:Integration`	13
`ait:Provider`	12
`ait:Task`	10
`ait:License`	6
`ait:ImageGenerationModel`	5
`ait:OpenLicense`	5
`ait:Benchmark`	5
`ait:Modality`	4

Properties not asserted directly in the data: `ait:develops`.

Example triples — a tool, a model and a benchmark result:

```
aid:GitHubCopilot a ait:AITool ;
    rdfs:label "GitHub Copilot" ;
    ait:providedBy aid:GitHub ;
    ait:releaseDate "2022-06-21"^^xsd:date ;
    ait:poweredBy aid:GPT41 , aid:GPT5 , aid:ClaudeSonnet45 , aid:Gemini25Pro ;
    ait:supportsTask aid:CodeGeneration , aid:CodeCompletion , aid:CodeReview ;
    ait:integratesWith aid:VSCode , aid:JetBrains , aid:GitHubPlatform , aid:CLI ;
    ait:hasPricingPlan aid:CopilotFree , aid:CopilotPro , aid:CopilotProPlus , aid:CopilotBusiness ;
    ait:competesWith aid:Cursor .

aid:ClaudeSonnet45 a ait:LanguageModel ;
    rdfs:label "Claude Sonnet 4.5" ;
    ait:developedBy aid:Anthropic ;
    ait:releaseDate "2025-09-29"^^xsd:date ;
    ait:contextWindow 200000 ;
    ait:inputModality aid:Text , aid:Image ;
    ait:outputModality aid:Text ;
```

```
    ait:hasLicense aid:Proprietary ;
    ait:derivedFrom aid:ClaudeSonnet4 .

aid:R_SWE_ClaudeSonnet45 a ait:BenchmarkResult ;
    ait:evaluatedModel aid:ClaudeSonnet45 ;
    ait:onBenchmark aid:SWEbenchVerified ;
    ait:score 77.2 .
```

A second file, `data/ai-tools-data-invalid.ttl`, contains seven intentionally broken resources used to test the SHACL shapes (Section 5).

4 SPARQL queries

All queries are stored in the `queries/` directory and were executed with RDFLib over the union of the ontology and the data (the ontology is needed for `rdfs:subClassOf*` paths). Results below are produced automatically by the build script. The following prefixes are declared in every query file and are omitted from the listings below for brevity:

```
PREFIX ait:  <https://otavlna.github.io/4iz444-ai-tools-kg/ontology#>
PREFIX aid:  <https://otavlna.github.io/4iz444-ai-tools-kg/data/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
```

CQ1 — Coding tools available in Visual Studio Code

A simple conjunctive query combining a task, an integration and a property path (`ait:providedBy/rdfs:label`) to the provider's name.

```
SELECT ?tool ?provider
WHERE {
  ?t a ait:AITool ;
     rdfs:label ?tool ;
     ait:supportsTask aid:CodeGeneration ;
     ait:integratesWith aid:VSCode ;
     ait:providedBy/rdfs:label ?provider .
}
ORDER BY ?tool
```

tool	provider
Claude Code	Anthropic
Gemini Code Assist	Google
GitHub Copilot	GitHub
OpenAI Codex	OpenAI

4 rows.

Four tools satisfy both conditions. Cursor is not listed because it is a stand-alone editor (a desktop application) rather than a VS Code extension, and chat assistants such as ChatGPT generate code but do not integrate into the IDE.

CQ2 — Models with image input and text output

The class is matched with `a/rdfs:subClassOf* ait:AIModel`, so the query works for instances of any subclass. `OPTIONAL` + `COUNT(DISTINCT)` adds the number of tools using the model (0 for models that are in the graph only as predecessors).

```
SELECT ?model ?developer (COUNT(DISTINCT ?tool) AS ?usedInTools)
WHERE {
  ?m a/rdfs:subClassOf* ait:AIModel ;
     rdfs:label ?model ;
     ait:inputModality aid:Image ;
     ait:outputModality aid:Text ;
     ait:developedBy/rdfs:label ?developer .
  OPTIONAL { ?tool ait:poweredBy ?m }
}
GROUP BY ?model ?developer
ORDER BY DESC(?usedInTools) ?model
```

model	developer	usedInTools
Claude Sonnet 4.5	Anthropic	5
GPT-5	OpenAI	5

model	developer	usedInTools
Gemini 2.5 Pro	Google	5
Claude Opus 4.1	Anthropic	2
Mistral Small 3.1	Mistral AI	2
GPT-4.1	OpenAI	1
GPT-4o	OpenAI	1
Gemini 2.5 Flash	Google	1
Gemma 3 27B	Google	1
Llama 4 Maverick	Meta	1
Mistral Medium 3	Mistral AI	1
o3	OpenAI	1
Claude 3.5 Sonnet	Anthropic	0
Claude 3.7 Sonnet	Anthropic	0
Claude Sonnet 4	Anthropic	0
Gemini 2.0 Flash	Google	0

16 rows.

Multimodal input has become the norm: 16 of the 22 language models accept images. The most widely used ones are the current flagships of the three big labs (Claude Sonnet 4.5, GPT-5, Gemini 2.5 Pro), each used in five tools — their own and those of third parties.

CQ3 — Average price of paid plans per provider

Because one plan can be linked to several tools of the same provider, the inner sub-query first selects `DISTINCT` (provider, plan) pairs; the outer query aggregates with `COUNT`, `AVG`, `MIN` and `MAX` and rounds the average to two decimals.

```
SELECT ?provider ?currency (COUNT(?plan) AS ?paidPlans)
       (ROUND(AVG(?price) * 100) / 100 AS ?avgPrice)
       (MIN(?price) AS ?minPrice) (MAX(?price) AS ?maxPrice)
WHERE {
  {
    SELECT DISTINCT ?provider ?plan ?price ?currency
    WHERE {
      ?tool ait:providedBy/rdfs:label ?provider ;
            ait:hasPricingPlan ?plan .
      ?plan ait:monthlyPrice ?price ;
            ait:currency ?currency .
      FILTER (?price > 0)
    }
  }
}
GROUP BY ?provider ?currency
ORDER BY DESC(?avgPrice)
```

provider	currency	paidPlans	avgPrice	minPrice	maxPrice
Perplexity AI	USD	2	110	20.00	200.00
Google	USD	3	97.59	19.99	249.99
Anysphere	USD	3	86.67	20.00	200.00
OpenAI	USD	3	83.33	20.00	200.00
Midjourney, Inc.	USD	4	55	10.00	120.00
Anthropic	USD	3	50	20.00	100.00
GitHub	USD	3	22.67	10.00	39.00

provider	currency	paidPlans	avgPrice	minPrice	maxPrice
Mistral AI	EUR	1	14.99	14.99	14.99

8 rows.

The averages are dominated by premium tiers: Perplexity, Google, Anysphere (Cursor) and OpenAI each have a 200–250 USD tier, while the entry-level paid tier is almost universally 20 USD. GitHub has the lowest average price of paid plans (22.67 USD) and shares the cheapest entry tier (10 USD) with Midjourney. Le Chat is priced in EUR, which is why the currency is part of the grouping — averaging prices in different currencies would be meaningless.

CQ4 — Best open-weights models on MMLU

The licence is matched by its class (`a ait:OpenLicense`), the benchmark result is joined through the n-ary `BenchmarkResult` node.

```
SELECT ?model ?license ?parametersB ?score
WHERE {
  ?m ait:hasLicense ?l ;
     rdfs:label ?model .
  ?l a ait:OpenLicense ;
     rdfs:label ?license .
  OPTIONAL { ?m ait:parameterCount ?parametersB }
  ?r ait:evaluatedModel ?m ;
     ait:onBenchmark aid:MMLU ;
     ait:score ?score .
}
ORDER BY DESC(?score)
```

model	license	parametersB	score
DeepSeek-R1	MIT License	671.0	90.8
gpt-oss-120b	Apache License 2.0	117.0	90.0
Llama 3.1 405B	Llama 3.1 Community License	405.0	88.6
DeepSeek-V3	MIT License	671.0	88.5
Qwen3-235B-A22B	Apache License 2.0	235.0	87.8
Llama 4 Maverick	Llama 4 Community License	400.0	85.5
Mistral Small 3.1	Apache License 2.0	24.0	80.6
Gemma 3 27B	Gemma Terms of Use	27.0	78.6

8 rows.

Openly licensed models reach the level of closed models on MMLU (GPT-4.1: 90.2, GPT-4o: 88.7). Model size helps but is not decisive: the 117B gpt-oss-120b scores nearly as well as the 671B DeepSeek-R1, and the small Mistral Small 3.1 (24B) and Gemma 3 (27B) are only ~10 points behind models fifteen times larger.

CQ5 — Number of tools per task

`GROUP BY` with `COUNT` and `GROUP_CONCAT`; the sub-query with `ORDER BY` makes the concatenated lists alphabetical and the results reproducible.

```
SELECT ?task (COUNT(?t) AS ?toolCount)
       (GROUP_CONCAT(?tool; separator=", ") AS ?tools)
WHERE {
  {
    SELECT ?task ?t ?tool
    WHERE {
      ?t ait:supportsTask ?tk ;
         rdfs:label ?tool .
      ?tk rdfs:label ?task .
    }
    ORDER BY ?tool
  }
```

```
}
GROUP BY ?task
ORDER BY DESC(?toolCount) ?task
```

task	toolCount	tools
Code generation	12	ChatGPT, Claude, Claude Code, Cursor, DeepSeek Chat, Gemini, Gemini Code Assist, GitHub Copilot, Le Chat, Ollama, OpenAI Codex, Qwen Chat
Conversation	9	ChatGPT, Claude, DeepSeek Chat, Gemini, Le Chat, Meta AI, Ollama, Perplexity, Qwen Chat
Text generation	8	ChatGPT, Claude, DeepSeek Chat, Gemini, Le Chat, Meta AI, Ollama, Qwen Chat
Web search	6	ChatGPT, Claude, DeepSeek Chat, Gemini, Le Chat, Perplexity
Code review	5	Claude Code, Cursor, Gemini Code Assist, GitHub Copilot, OpenAI Codex
Summarization	5	ChatGPT, Claude, Gemini, Le Chat, Perplexity
Image generation	4	ChatGPT, Gemini, Meta AI, Midjourney
Code completion	3	Cursor, Gemini Code Assist, GitHub Copilot
Data analysis	2	ChatGPT, Claude

9 rows.

Code generation is the most common capability (12 of 15 tools) — practically every general-purpose assistant can write code. Only specialised coding tools offer code review and inline code completion. The task *Knowledge and reasoning* does not appear because it is used only to classify benchmarks, not tools.

CQ6 — Tools built on models of other providers

The key condition is `FILTER (?tp != ?mp)`: the provider of the tool differs from the developer of the model.

```
SELECT ?tool ?toolProvider
       (COUNT(DISTINCT ?m) AS ?externalModels)
       (GROUP_CONCAT(DISTINCT ?modelProvider; separator=", ") AS ?dependsOn)
WHERE {
  {
    SELECT ?tool ?toolProvider ?m ?modelProvider
    WHERE {
      ?t a ait:AITool ;
         rdfs:label ?tool ;
         ait:providedBy ?tp ;
         ait:poweredBy ?m .
      ?tp rdfs:label ?toolProvider .
      ?m ait:developedBy ?mp .
      ?mp rdfs:label ?modelProvider .
      FILTER (?tp != ?mp)
    }
    ORDER BY ?modelProvider
  }
}
GROUP BY ?tool ?toolProvider
ORDER BY DESC(?externalModels) ?tool
```

tool	toolProvider	externalModels	dependsOn
Ollama	Ollama	6	Alibaba Cloud, DeepSeek, Google, Meta, Mistral AI, OpenAI
GitHub Copilot	GitHub	4	Anthropic, Google, OpenAI
Cursor	Anysphere	3	Anthropic, Google, OpenAI
Perplexity	Perplexity AI	3	Anthropic, Google, OpenAI

4 rows.

Four tools are pure “aggregators” without in-house models. GitHub Copilot, Cursor and Perplexity all depend on the same three labs (OpenAI, Anthropic, Google), which illustrates how concentrated the model layer of the market is. Ollama, a tool for running

models locally, depends on six providers, all of them publishing open weights.

CQ7 — Model lineage

The transitive relation `ait:derivedFrom` is followed with the property path `ait:derivedFrom+`. The difference of two `xsd:date` values is an `xsd:duration` (e.g. `P130D`), from which the number of days is extracted with `REPLACE` and cast to an integer.

```
SELECT ?model ?ancestor ?ancestorReleased ?daysBefore
WHERE {
  VALUES ?m { aid:ClaudeSonnet45 aid:GPT5 }
  ?m rdfs:label ?model ;
     ait:releaseDate ?released ;
     ait:derivedFrom+ ?a .
  ?a rdfs:label ?ancestor ;
     ait:releaseDate ?ancestorReleased .
  BIND (?released - ?ancestorReleased AS ?diff)
  BIND (xsd:integer(REPLACE(STR(?diff), "^P([0-9]+)D$", "$1")) AS ?daysBefore)
}
ORDER BY ?model ?ancestorReleased
```

model	ancestor	ancestorReleased	daysBefore
Claude Sonnet 4.5	Claude 3.5 Sonnet	2024-06-20	466
Claude Sonnet 4.5	Claude 3.7 Sonnet	2025-02-24	217
Claude Sonnet 4.5	Claude Sonnet 4	2025-05-22	130
GPT-5	GPT-4o	2024-05-13	451
GPT-5	GPT-4.1	2025-04-14	115
GPT-5	o3	2025-04-16	113

6 rows.

The data assert only direct predecessors (Sonnet 4.5 → Sonnet 4 → 3.7 Sonnet → 3.5 Sonnet), but the property path returns the whole chain. Both labs release a new flagship generation roughly every 4 months (113–130 days between GPT-5 and o3/GPT-4.1 and between Claude Sonnet 4.5 and Sonnet 4).

CQ8 — Largest context window per provider

A sub-query computes the maximum per provider; it is joined back to the models to find which model(s) reach it. Ties are aggregated with `GROUP_CONCAT`.

```
SELECT ?provider ?contextWindow (COUNT(?m) AS ?modelCount)
       (GROUP_CONCAT(?model; separator=", ") AS ?models)
WHERE {
  {
    SELECT ?p (MAX(?cw) AS ?contextWindow)
    WHERE {
      ?x a ait:LanguageModel ;
         ait:developedBy ?p ;
         ait:contextWindow ?cw .
    }
    GROUP BY ?p
  }
  {
    SELECT ?p ?m ?model ?contextWindow
    WHERE {
      ?m a ait:LanguageModel ;
         ait:developedBy ?p ;
         ait:contextWindow ?contextWindow ;
         rdfs:label ?model .
    }
    ORDER BY ?model
  }
  ?p rdfs:label ?provider .
}
GROUP BY ?provider ?contextWindow
ORDER BY DESC(?contextWindow) ?provider
```

provider	contextWindow	modelCount	models
Google	1048576	3	Gemini 2.0 Flash, Gemini 2.5 Flash, Gemini 2.5 Pro
OpenAI	1047576	1	GPT-4.1
Meta	1000000	1	Llama 4 Maverick
Mistral AI	256000	1	Codestral 25.01
Anthropic	200000	5	Claude 3.5 Sonnet, Claude 3.7 Sonnet, Claude Opus 4.1, Claude Sonnet 4, Claude Sonnet 4.5
Alibaba Cloud	131072	1	Qwen3-235B-A22B
DeepSeek	128000	2	DeepSeek-R1, DeepSeek-V3

7 rows.

Google, OpenAI (GPT-4.1) and Meta (Llama 4 Maverick) offer context windows of about one million tokens. All Anthropic models in the graph share the same 200k window, so five models tie for the maximum.

CQ9 — Providers offering both text and image generation

Two independent tool patterns for the same provider (`?t1`, `?t2`) express "the provider offers both"; an `OPTIONAL` block looks for an image model developed by the same provider.

```
SELECT ?provider
       (GROUP_CONCAT(DISTINCT ?imageModel; separator=", ") AS ?imageModels)
WHERE {
  ?p a ait:Provider ;
     rdfs:label ?provider .
  ?t1 ait:providedBy ?p ; ait:supportsTask aid:TextGeneration .
  ?t2 ait:providedBy ?p ; ait:supportsTask aid:ImageGeneration .
  OPTIONAL {
    ?t2 ait:poweredBy ?im .
    ?im a ait:ImageGenerationModel ;
        ait:developedBy ?p ;
        rdfs:label ?imName .
  }
  BIND (COALESCE(?imName, "(no in-house image model)") AS ?imageModel)
}
GROUP BY ?provider
ORDER BY ?provider
```

provider	imageModels
Google	Imagen 4
Meta	(no in-house image model)
OpenAI	gpt-image-1

3 rows.

OpenAI and Google cover both tasks with their own image models (gpt-image-1, Imagen 4). Meta AI also generates images, but no Meta image model is part of the graph — the query reveals a gap in the data that could be filled in a next version. Midjourney is not listed because it only generates images.

CQ10 — Free tools running on openly licensed models

Joins tools, pricing plans with price 0 and models whose licence is an `ait:OpenLicense`.

```
SELECT ?tool ?freePlan (COUNT(?m) AS ?openModelCount)
       (GROUP_CONCAT(?model; separator=", ") AS ?openModels)
WHERE {
  {
    SELECT DISTINCT ?tool ?freePlan ?m ?model
    WHERE {
      ?t a ait:AITool ;
         rdfs:label ?tool ;
         ait:hasPricingPlan ?plan ;
```

```
        ait:poweredBy ?m .
      ?plan ait:monthlyPrice 0.00 ;
            rdfs:label ?freePlan .
      ?m ait:hasLicense/a ait:OpenLicense ;
         rdfs:label ?model .
    }
    ORDER BY ?model
  }
}
GROUP BY ?tool ?freePlan
ORDER BY DESC(?openModelCount) ?tool
```

tool	freePlan	openModelCount	openModels
Ollama	Ollama (local, free)	6	DeepSeek-R1, Gemma 3 27B, Llama 3.1 405B, Mistral Small 3.1, Qwen3-235B-A22B, gpt-oss-120b
DeepSeek Chat	DeepSeek Chat (free)	2	DeepSeek-R1, DeepSeek-V3
Le Chat	Le Chat Free	1	Mistral Small 3.1
Meta AI	Meta AI (free)	1	Llama 4 Maverick
Qwen Chat	Qwen Chat (free)	1	Qwen3-235B-A22B

5 rows.

Five tools can be used free of charge with an open-weights model. Ollama stands out with six such models, because it runs them locally on the user's hardware; DeepSeek Chat, Meta AI and Qwen Chat are hosted chat applications of providers that publish the weights of their models.

Q11 — Providers' best scores compared with the benchmark average

An analytical query with two levels of aggregation: the sub-query computes the average score per benchmark (only benchmarks with at least 5 results are considered), the outer query takes the best (`MAX`) model of each provider and computes the difference to the average.

```
SELECT ?benchmark ?provider (MAX(?score) AS ?bestScore) ?benchmarkAvg
       (ROUND((MAX(?score) - ?benchmarkAvg) * 10) / 10 AS ?diffToAvg)
WHERE {
  {
    SELECT ?b (ROUND(AVG(?s) * 10) / 10 AS ?benchmarkAvg) (COUNT(?r2) AS ?n)
    WHERE { ?r2 ait:onBenchmark ?b ; ait:score ?s . }
    GROUP BY ?b
  }
  ?r ait:onBenchmark ?b ;
     ait:score ?score ;
     ait:evaluatedModel/ait:developedBy/rdfs:label ?provider .
  ?b rdfs:label ?benchmark .
  FILTER (?n >= 5)
}
GROUP BY ?benchmark ?provider ?benchmarkAvg
ORDER BY ?benchmark DESC(?bestScore)
```

benchmark	provider	bestScore	benchmarkAvg	diffToAvg
AIME 2025	OpenAI	94.6	88.5	6.1
AIME 2025	Anthropic	87.0	88.5	-1.5
AIME 2025	Google	86.7	88.5	-1.8
AIME 2025	Alibaba Cloud	81.5	88.5	-7
GPQA Diamond	Google	86.4	76.7	9.7
GPQA Diamond	OpenAI	85.7	76.7	9
GPQA Diamond	Anthropic	83.4	76.7	6.7
GPQA Diamond	DeepSeek	71.5	76.7	-5.2

benchmark	provider	bestScore	benchmarkAvg	diffToAvg
GPQA Diamond	Alibaba Cloud	71.1	76.7	-5.6
GPQA Diamond	Meta	69.8	76.7	-6.9
HumanEval	Anthropic	92.0	89.2	2.8
HumanEval	OpenAI	90.2	89.2	1
HumanEval	Meta	89.0	89.2	-0.2
HumanEval	Mistral AI	88.4	89.2	-0.8
MMLU	DeepSeek	90.8	87.1	3.7
MMLU	OpenAI	90.2	87.1	3.1
MMLU	Anthropic	88.7	87.1	1.6
MMLU	Meta	88.6	87.1	1.5
MMLU	Alibaba Cloud	87.8	87.1	0.7
MMLU	Mistral AI	80.6	87.1	-6.5
MMLU	Google	78.6	87.1	-8.5
SWE-bench Verified	Anthropic	77.2	64.5	12.7
SWE-bench Verified	OpenAI	74.9	64.5	10.4
SWE-bench Verified	Google	63.8	64.5	-0.7
SWE-bench Verified	DeepSeek	49.2	64.5	-15.3

25 rows.

The spread is largest on the agentic coding benchmark SWE-bench Verified, where Anthropic's best model is 12.7 points above the average and DeepSeek 15.3 points below it. On the older MMLU, the results are saturated — the best models of five providers are within 4 points of the average, and only the smaller models (Mistral Small 3.1, Gemma 3 27B) fall clearly behind.

Q12 — Analysis of model names (string functions)

Uses `REGEX`, `REPLACE` with a capturing group, `STRBEFORE` and `UCASE` to split names such as "Claude Sonnet 4.5" into a family ("CLAUDE SONNET") and a version ("4.5"), and then aggregates releases per family with `HAVING (COUNT(?m) > 1)`.

```
SELECT ?family (COUNT(?m) AS ?releases)
       (GROUP_CONCAT(?version; separator=" -> ") AS ?versions)
       (MIN(?released) AS ?firstRelease) (MAX(?released) AS ?lastRelease)
WHERE {
  {
    SELECT ?m ?released ?version ?family
    WHERE {
      ?m a/rdfs:subClassOf* ait:AIModel ;
         rdfs:label ?name ;
         ait:releaseDate ?released .
      FILTER (REGEX(?name, "[0-9]"))
      BIND (REPLACE(?name, "^[^0-9]*?([0-9]+(\\.[0-9]+)?).*$", "$1") AS ?version)
      BIND (UCASE(REPLACE(STRBEFORE(?name, ?version), "[\\s-]*[Vv]?[\\s-]*$", "")) AS ?rawFamily)
      BIND (IF(?rawFamily = "", "O-SERIES", ?rawFamily) AS ?family)
    }
    ORDER BY ?released
  }
}
GROUP BY ?family
HAVING (COUNT(?m) > 1)
ORDER BY DESC(?releases) ?family
```

family	releases	versions	firstRelease	lastRelease
GEMINI	3	2.0 -> 2.5 -> 2.5	2024-12-11	2025-04-17
GPT	3	4 -> 4.1 -> 5	2024-05-13	2025-08-07

family	releases	versions	firstRelease	lastRelease
CLAUDE	2	3.5 -> 3.7	2024-06-20	2025-02-24
CLAUDE SONNET	2	4 -> 4.5	2025-05-22	2025-09-29
LLAMA	2	3.1 -> 4	2024-07-23	2025-04-05
MIDJOURNEY	2	6.1 -> 7	2024-07-30	2025-04-03

6 rows.

The query recovers model families purely from naming conventions. It also uncovers a change of naming scheme: Anthropic's "Claude 3.5 Sonnet" / "Claude 3.7 Sonnet" became "Claude Sonnet 4" / "Claude Sonnet 4.5", so the string analysis splits one product line into two families — the explicit `ait:derivedFrom` relation (CQ7) is the more reliable way to express lineage.

Q13 — Competitor network (symmetric property, EXISTS)

The relation `competesWith` is declared symmetric but is stated only in one direction in the data; the alternative path `ait:competesWith|^ait:competesWith` follows both directions without reasoning. `EXISTS` inside `IF` tests whether two competitors share an underlying model.

```
SELECT ?tool (COUNT(?c) AS ?competitors)
       (SUM(?shares) AS ?sharingModel)
       (GROUP_CONCAT(?competitor; separator=", ") AS ?competitorNames)
WHERE {
  {
    SELECT ?tool ?c ?competitor ?shares
    WHERE {
      ?t a ait:AITool ; rdfs:label ?tool .
      OPTIONAL {
        ?t ait:competesWith|^ait:competesWith ?c .
        ?c rdfs:label ?competitorName .
      }
      BIND (COALESCE(?competitorName, "—") AS ?competitor)
      # 1 if the tool and its competitor use at least one common model, otherwise 0
      BIND (IF(BOUND(?c) && EXISTS { ?t ait:poweredBy ?x . ?c ait:poweredBy ?x . }, 1, 0) AS ?shares)
    }
    ORDER BY ?competitor
  }
}
GROUP BY ?tool
ORDER BY DESC(?competitors) ?tool
```

tool	competitors	sharingModel	competitorNames
ChatGPT	5	1	Claude, DeepSeek Chat, Gemini, Le Chat, Perplexity
Gemini	4	1	ChatGPT, Claude, Meta AI, Perplexity
GitHub Copilot	4	4	Claude Code, Cursor, Gemini Code Assist, OpenAI Codex
Claude	3	0	ChatGPT, Gemini, Le Chat
Claude Code	3	2	Cursor, GitHub Copilot, OpenAI Codex
Cursor	3	3	Claude Code, GitHub Copilot, OpenAI Codex
OpenAI Codex	3	2	Claude Code, Cursor, GitHub Copilot
DeepSeek Chat	2	0	ChatGPT, Qwen Chat
Le Chat	2	0	ChatGPT, Claude
Perplexity	2	2	ChatGPT, Gemini
Gemini Code Assist	1	1	GitHub Copilot
Meta AI	1	0	Gemini
Qwen Chat	1	0	DeepSeek Chat
Midjourney	0	0	-

tool	competitors	sharingModel	competitorNames
Ollama	0	0	-

15 rows.

ChatGPT has the most direct competitors. In the coding segment (Copilot, Cursor, Claude Code, Codex) competitors almost always share models — they compete on the product, not on the model. In the chat segment, competitors mostly use their own models (Claude, Le Chat, DeepSeek Chat share none with their competitors).

Q14 — Models used in tools but lacking benchmark results (negation)

`FILTER NOT EXISTS` finds language models that are used by at least one tool but have no `BenchmarkResult` — a data-quality / data-gap check.

```
SELECT ?model ?developer (COUNT(DISTINCT ?t) AS ?usedInTools)
WHERE {
  ?m a ait:LanguageModel ;
     rdfs:label ?model ;
     ait:developedBy/rdfs:label ?developer .
  ?t ait:poweredBy ?m .
  FILTER NOT EXISTS { ?r ait:evaluatedModel ?m }
}
GROUP BY ?model ?developer
ORDER BY DESC(?usedInTools) ?model
```

model	developer	usedInTools
Gemini 2.5 Flash	Google	1
Mistral Medium 3	Mistral AI	1

2 rows.

Two models in production use lack benchmark data in the graph and are candidates for the next data update.

Q15 — CONSTRUCT: materialising tool dependencies

A `CONSTRUCT` query derives new knowledge: a direct link `schema:isBasedOn` from a tool to every external provider whose model it uses. The resulting graph (stored in `results/q15-construct-tool-dependency.ttl`) could be added to the knowledge graph to answer dependency questions without the two-step path.

```
PREFIX schema: <https://schema.org/>
CONSTRUCT {
  ?t schema:isBasedOn ?mp .
}
WHERE {
  ?t ait:providedBy ?tp ;
     ait:poweredBy/ait:developedBy ?mp .
  FILTER (?tp != ?mp)
}
```

15 triples constructed:

```
aid:Cursor schema:isBasedOn aid:Anthropic,
        aid:Google,
        aid:OpenAI .

aid:GitHubCopilot schema:isBasedOn aid:Anthropic,
        aid:Google,
        aid:OpenAI .

aid:Ollama schema:isBasedOn aid:Alibaba,
        aid:DeepSeek,
        aid:Google,
        aid:Meta,
        aid:MistralAI,
        aid:OpenAI .
```

```
aid:Perplexity schema:isBasedOn aid:Anthropic,
        aid:Google,
        aid:OpenAI .
```

5 SHACL validation

The shapes are in `shacl/shapes.ttl` and were validated with pySHACL. Six node shapes cover all main classes; one of them uses a SPARQL-based constraint.

S1 AIToolShape

Every tool has a name (`xsd:string`), exactly one provider (`sh:minCount 1` + `sh:maxCount 1`, value of class `ait:Provider`), at least one model of class `ait:AIModel`, at least one task and at least one pricing plan.

```
aish:AIToolShape
    a sh:NodeShape ;
    sh:targetClass ait:AITool ;
    sh:property [
        sh:path rdfs:label ;
        sh:minCount 1 ;
        sh:datatype xsd:string ;
        sh:message "An AI tool must have a name (rdfs:label of type xsd:string)." ;
    ] , [
        sh:path ait:providedBy ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:class ait:Provider ;
        sh:message "An AI tool must have exactly one provider of type ait:Provider." ;
    ] , [
        sh:path ait:poweredBy ;
        sh:minCount 1 ;
        sh:class ait:AIModel ;
        sh:message "An AI tool must be powered by at least one ait:AIModel." ;
    ] , [
        sh:path ait:supportsTask ;
        sh:minCount 1 ;
        sh:class ait:Task ;
        sh:message "An AI tool must support at least one ait:Task." ;
    ] , [
        sh:path ait:hasPricingPlan ;
        sh:minCount 1 ;
        sh:class ait:PricingPlan ;
        sh:message "An AI tool must have at least one pricing plan." ;
    ] .
```

S2 AIModelShape

Every model has exactly one developer, exactly one release date of datatype `xsd:date`, exactly one licence and at least one input and output modality. The context window and parameter count are optional, but when present they must be single positive numbers of the right datatype (`sh:minExclusive 0`).

```
aish:AIModelShape
    a sh:NodeShape ;
    sh:targetClass ait:AIModel ;
    sh:property [
        sh:path ait:developedBy ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:class ait:Provider ;
        sh:message "An AI model must be developed by exactly one ait:Provider." ;
    ] , [
        sh:path ait:releaseDate ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:datatype xsd:date ;
        sh:message "An AI model must have exactly one release date of type xsd:date." ;
    ] , [
        sh:path ait:hasLicense ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:class ait:License ;
        sh:message "An AI model must have exactly one licence." ;
    ] , [
        sh:path ait:inputModality ;
        sh:minCount 1 ;
        sh:class ait:Modality ;
```

```
        sh:message "An AI model must declare at least one input modality." ;
    ] , [
        sh:path ait:outputModality ;
        sh:minCount 1 ;
        sh:class ait:Modality ;
        sh:message "An AI model must declare at least one output modality." ;
    ] , [
        sh:path ait:contextWindow ;
        sh:maxCount 1 ;
        sh:datatype xsd:integer ;
        sh:minExclusive 0 ;
        sh:message "The context window must be a single positive integer (number of tokens)." ;
    ] , [
        sh:path ait:parameterCount ;
        sh:maxCount 1 ;
        sh:datatype xsd:decimal ;
        sh:minExclusive 0 ;
        sh:message "The parameter count must be a single positive decimal (billions of parameters)." ;
    ] .
```

S3 LanguageModelShape and ImageGenerationModelShape

Subclass-specific rules: a language model must state its context window and must output text (`sh:hasValue aid:Text`); an image generation model must output images.

```
aish:LanguageModelShape
    a sh:NodeShape ;
    sh:targetClass ait:LanguageModel ;
    sh:property [
        sh:path ait:contextWindow ;
        sh:minCount 1 ;
        sh:message "A language model must state its context window." ;
    ] , [
        sh:path ait:outputModality ;
        sh:hasValue aid:Text ;
        sh:message "A language model must produce text output." ;
    ] .
```

```
aish:ImageGenerationModelShape
    a sh:NodeShape ;
    sh:targetClass ait:ImageGenerationModel ;
    sh:property [
        sh:path ait:outputModality ;
        sh:hasValue aid:Image ;
        sh:message "An image generation model must produce image output." ;
    ] .
```

S4 BenchmarkResultShape

Every result refers to exactly one model and one benchmark and has exactly one decimal score in the interval [0, 100] (`sh:minInclusive`, `sh:maxInclusive`).

```
aish:BenchmarkResultShape
    a sh:NodeShape ;
    sh:targetClass ait:BenchmarkResult ;
    sh:property [
        sh:path ait:evaluatedModel ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:class ait:AIModel ;
        sh:message "A benchmark result must refer to exactly one evaluated AI model." ;
    ] , [
        sh:path ait:onBenchmark ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:class ait:Benchmark ;
        sh:message "A benchmark result must refer to exactly one benchmark." ;
    ] , [
        sh:path ait:score ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:datatype xsd:decimal ;
        sh:minInclusive 0 ;
        sh:maxInclusive 100 ;
```

```
        sh:message "A benchmark score must be a single decimal between 0 and 100." ;
    ] .
```

S5 PricingPlanShape

Every plan has a name and exactly one non-negative decimal price; the currency must be one of the supported codes (`sh:in ("USD" "EUR")`).

```
aish:PricingPlanShape
    a sh:NodeShape ;
    sh:targetClass ait:PricingPlan ;
    sh:property [
        sh:path rdfs:label ;
        sh:minCount 1 ;
        sh:message "A pricing plan must have a name." ;
    ] , [
        sh:path ait:monthlyPrice ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:datatype xsd:decimal ;
        sh:minInclusive 0 ;
        sh:message "A pricing plan must have exactly one non-negative monthly price (xsd:decimal)." ;
    ] , [
        sh:path ait:currency ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:in ( "USD" "EUR" ) ;
        sh:message "The currency must be one of the supported ISO 4217 codes: USD, EUR." ;
    ] .
```

S6 ModelLineageShape (SPARQL constraint)

A model cannot have been released before a model it is derived from. This temporal rule compares values of two different nodes and cannot be expressed with core SHACL constraints, so a `sh:SPARQLConstraint` is used; every row returned by the `SELECT` is a violation.

```
aish:ModelLineageShape
    a sh:NodeShape ;
    sh:targetClass ait:AIModel ;
    sh:sparql [
        a sh:SPARQLConstraint ;
        sh:message "Model {$this} (released {?date}) is derived from {?parent}, which was released later ({?
parentDate})." ;
        sh:prefixes aish: ;
        sh:select """
            SELECT $this ?parent ?date ?parentDate
            WHERE {
                $this ait:derivedFrom ?parent ;
                      ait:releaseDate ?date .
                ?parent ait:releaseDate ?parentDate .
                FILTER (?date < ?parentDate)
            }
        """ ;
    ] .
```

5.1 Validation results

Valid knowledge graph (`data/ai-tools-data.ttl`):

Conforms: `true` — 0 violation(s).

Knowledge graph + intentionally invalid data (`data/ai-tools-data-invalid.ttl`). The file contains a tool without a provider and pricing plan, a tool “powered by” a provider instead of a model, a model with a string release date, a negative context window, no licence and audio-only output, an image model producing only text, a benchmark result with a score of 112.5 and no benchmark, a plan with a negative price in CZK, and a model released before its predecessor:

Conforms: `false` — 13 violation(s).

Focus node	Shape	Path	Constraint	Message
aid:BadModel_NoImage	aish:ImageGenerationModelShape	ait:outputModality	sh:HasValueConstraintComponent	An image generation model must produce image output.
aid:BadModel_TimeTravel	aish:ModelLineageShape	(SPARQL)	sh:SPARQLConstraintComponent	Model aid:BadModel_TimeTravel (released 2024-01-01) is derived from aid:ClaudeSonnet45, which was released later (2025-09-29).
aid:BadModel_Types	aish:AIModelShape	ait:contextWindow	sh:MinExclusiveConstraintComponent	The context window must be a single positive integer (number of tokens).
aid:BadModel_Types	aish:AIModelShape	ait:hasLicense	sh:MinCountConstraintComponent	An AI model must have exactly one licence.
aid:BadModel_Types	aish:AIModelShape	ait:releaseDate	sh:DatatypeConstraintComponent	An AI model must have exactly one release date of type xsd:date.
aid:BadModel_Types	aish:LanguageModelShape	ait:outputModality	sh:HasValueConstraintComponent	A language model must produce text output.
aid:BadPlan_Price	aish:PricingPlanShape	ait:currency	sh:InConstraintComponent	The currency must be one of the supported ISO 4217 codes: USD, EUR.
aid:BadPlan_Price	aish:PricingPlanShape	ait:monthlyPrice	sh:MinInclusiveConstraintComponent	A pricing plan must have exactly one non-negative monthly price (xsd:decimal).
aid:BadResult_Score	aish:BenchmarkResultShape	ait:onBenchmark	sh:MinCountConstraintComponent	A benchmark result must refer to exactly one benchmark.
aid:BadResult_Score	aish:BenchmarkResultShape	ait:score	sh:MaxInclusiveConstraintComponent	A benchmark score must be a single decimal between 0 and 100.
aid:BadTool_NoProvider	aish:AIToolShape	ait:hasPricingPlan	sh:MinCountConstraintComponent	An AI tool must have at least one pricing plan.
aid:BadTool_NoProvider	aish:AIToolShape	ait:providedBy	sh:MinCountConstraintComponent	An AI tool must have exactly one provider of type ait:Provider.
aid:BadTool_WrongModel	aish:AIToolShape	ait:poweredBy	sh:ClassConstraintComponent	An AI tool must be powered by at least one ait:AIModel.

Every intentionally introduced error was detected by the expected shape and constraint component, and the valid data conform to all shapes, which shows that the shapes are neither too weak nor too strict for the real data.

6 Conclusion

We designed an ontology with 13 classes, 25 properties and a range of OWL axioms for a catalogue of AI tools, populated it with a knowledge graph of almost 1,000 triples, answered ten competency questions and five additional analytical queries in SPARQL, and validated the data with six SHACL shapes. The most valuable modelling decisions were the separation of tools from models and the modelling of benchmark results and pricing plans as separate nodes — they enable the most interesting analyses (dependencies between providers, price comparisons, comparison with benchmark averages).

Limitations and future work. The data are a manually created snapshot and will age quickly; benchmark scores are vendor-reported under different settings. A natural next step would be to populate the graph automatically from structured sources (e.g. Hugging Face model cards, Wikidata, public leaderboards) using YARRRML/RML mappings, to add API pricing per token, and to extend the `Task` concept into a SKOS hierarchy (e.g. *code completion* $\sqsubseteq$ *code generation*).

Appendix A — Repository and reproducibility

All files are available at https://github.com/otavlna/4iz444-ai-tools-kg:

Path	Content
`ontology/ai-tools.ttl`	ontology (OWL 2, Turtle)
`data/ai-tools-data.ttl`	knowledge graph
`data/ai-tools-data-invalid.ttl`	intentionally invalid data for SHACL testing
`queries/*.rq`	SPARQL queries CQ1–CQ10, Q11–Q15
`shacl/shapes.ttl`	SHACL shapes
`results/`	query results (CSV, Markdown, Turtle) and SHACL reports
`scripts/build.py`	build pipeline (parsing, reasoning, queries, SHACL, diagram, report)

Running `make all` re-creates all results, the diagram and this report from the source files.